

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

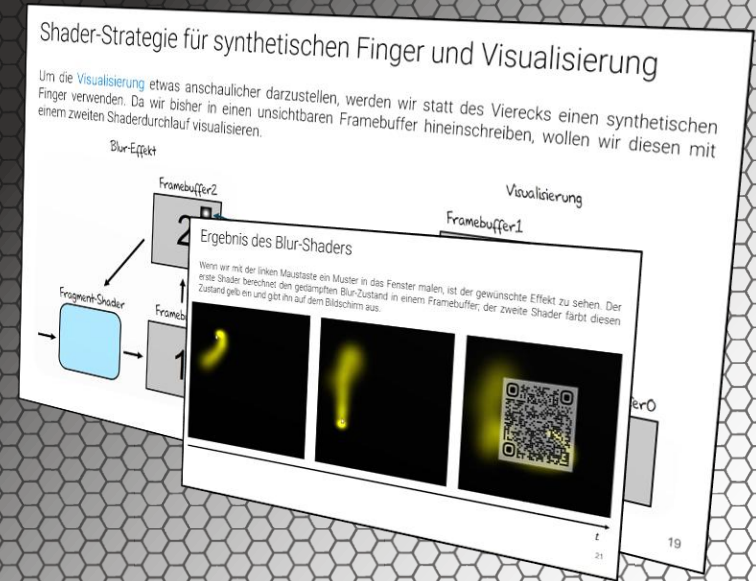
Sommersemester 2026



Realisierung einer einfachen Wasserdynamik in Java und im Shader

„Das Wasser nimmt jede Form an, in die man es gießt, und doch bleibt es sich selbst treu.“ (sinngemäß nach Katsushika Hokusai)

PD Prof. Dr. Marco Block-Berlitz



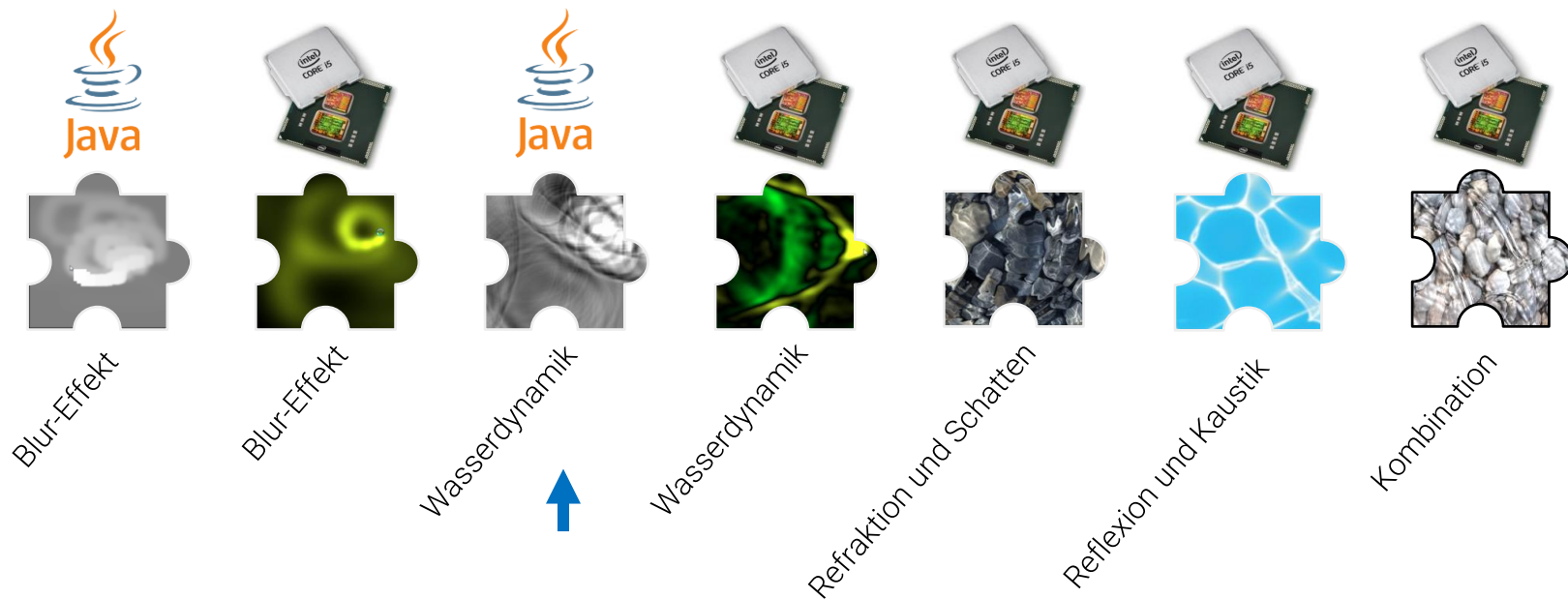
Ziele und Inhalt

- Projektetappen Interaktives Wasser
 - Projektstufe 3 (Java)
 - Motivation Wellengleichung
 - Wellengleichung in der Physik
 - Vereinfachte Schwingungsdynamik
 - Pseudocode
 - Implementierung in Java
 - Korrektur der Darstellung
 - Projektstufe 4 (Shader)
 - Shaderstrategie Wasserdynamik
 - Implementierung vereinfachte Wasserdynamik
 - Implementierung Visualisierung
- Fragestellungen zum Vorlesungsteil
 - Praktische Aufgaben



Interaktives Wasser – Projektetappen

Wasser hat in diesem Kontext schon immer eine besondere Anziehungskraft ausgeübt, symbolisiert es doch den Ursprung und das Grundbedürfnis des Lebens. Das folgende Projekt widmet sich dem Motto „Es war einmal das Wasser ...“ und wird Schritt für Schritt in die [Shaderprogrammierung mit GLSL](#) einführen und so – ganz nebenbei – die notwendigen mathematischen und physikalischen Konzepte behandeln. Beiläufig sehen wir instruktive Shaderbeispiele und machen uns mit GLSL vertraut.



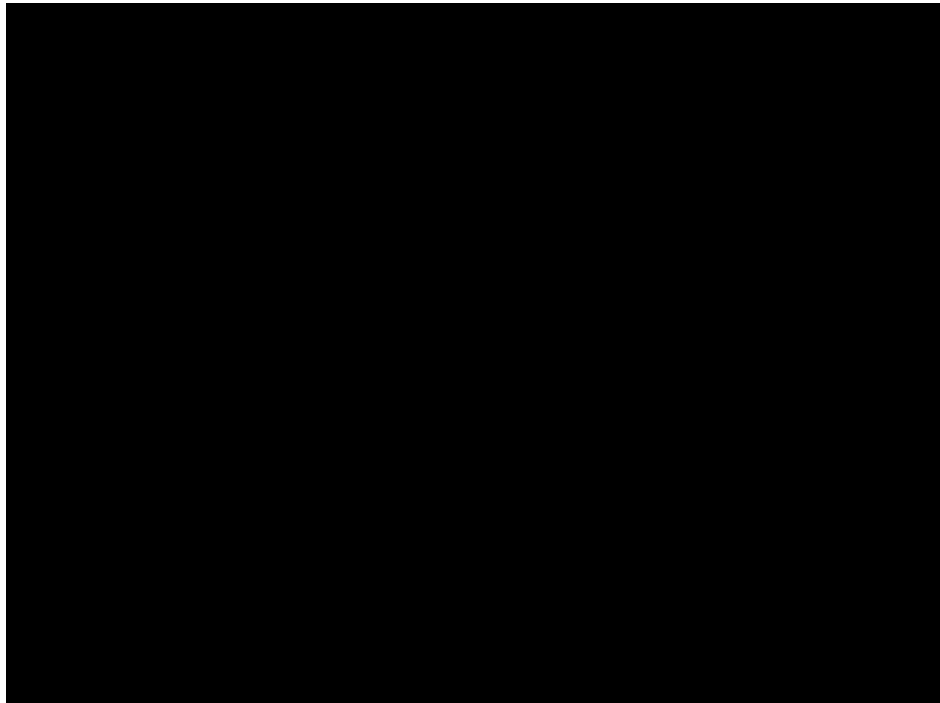


Bevor es losgeht, wollen wir noch einmal das Beispiel QuadGelb „unter die Lupe nehmen“

Gelbes Quad rotieren lassen (Java)

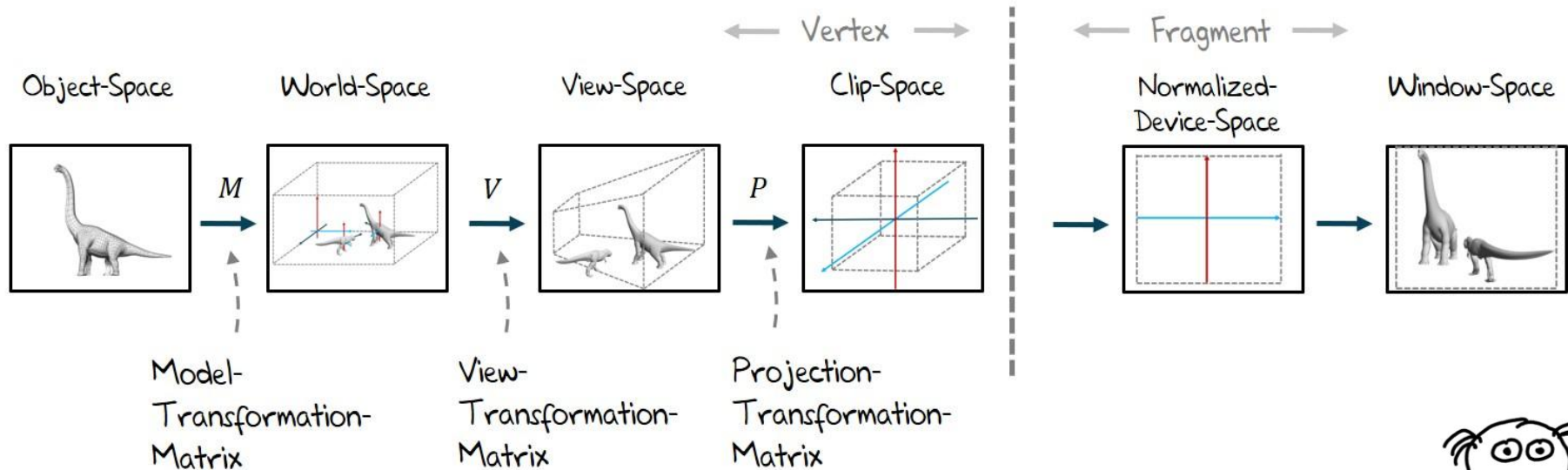
Im **renderLoop** wollen wir jetzt die Methode implementieren, die für die Visualisierung zuständig ist. In einer Schleife wird dabei das **Quad** rotiert und durch den Shader visualisiert:

```
public void renderLoop() {  
    prepareShader();  
  
    glEnable(GL_DEPTH_TEST);  
    long start = System.nanoTime();  
    while (!Display.isCloseRequested()) {  
        double now = (System.nanoTime() - start) / 1e9;  
        POGL.setClearColorClearDepth(0.0f, 0.0f, 0.0f);  
  
        glMatrixMode(GL_PROJECTION);  
        glLoadIdentity();  
        double r = (double)getHeight()/getWidth();  
        glFrustum(-1, 1, -r, r, 1, 20);  
        glMatrixMode(GL_MODELVIEW);  
        glLoadIdentity();  
        glTranslated(0, 0, -3);  
        glRotatef((float) now * 5.0f, 0, 0, 1);  
        glRotatef(10 * (float) now * 5.0f, 1, 0, 0);  
  
        POGL.renderViereck();  
        Display.update();  
    }  
}
```



Kombination zur MVP-Matrix

Hier sehen wir noch einmal, wie die Daten in den ersten drei Schritten die Model-View-Projection-Matrix durchlaufen.



Die Model-View-Projection-Matrix MVP in der Übersicht: $M = P_{persp} \cdot V \cdot M$

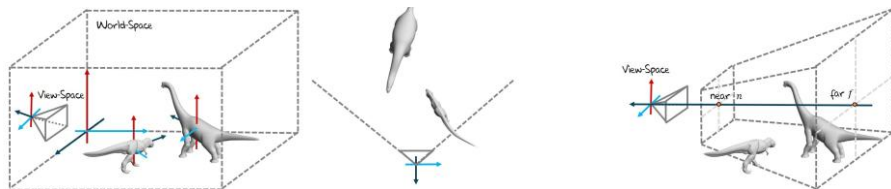


Gelbes Quad rotieren lassen und MVP-Matrix

Mit folgendem OpenGL-Code und dem zugehörigen Vertex-Shader bringen wir Bewegung ins Spiel:

```
glMatrixMode(GL_PROJECTION);  
glLoadIdentity();  
double r = (double)getHeight()/getWidth();  
glFrustum(-1, 1, -r, r, 1, 20);  
  
glMatrixMode(GL_MODELVIEW);  
glLoadIdentity();  
glTranslated(0, 0, -3);  
glRotatef((float) now * 5.0f, 0, 0, 1);  
glRotatef(10 * (float) now * 5.0f, 1, 0, 0);
```

```
void main() {  
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;  
}
```



Ohne die Trennung in Projektions- und Model-View-Matrix können wir die MVP-Matrix auch direkt angeben:

```
glLoadIdentity();  
double r = (double)getHeight()/getWidth();  
glFrustum(-1, 1, -r, r, 1, 20);  
glTranslated(0, 0, -3);  
glRotatef((float) now * 5.0f, 0, 0, 1);  
glRotatef(10 * (float) now * 5.0f, 1, 0, 0);
```

```
void main() {  
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;  
}
```

```
void main() {  
    gl_Position = gl_Projection * gl_ModelView * gl_Vertex;  
}
```

$$M = P_{persp} \cdot V \cdot M$$

MVP-Matrix in OpenGL und im Shader erweitern

Mit folgendem Vertex-Shader bringen wir Bewegung ins Spiel:

```
void main() {  
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;  
}
```

```
uniform vec3 uniform_scale_mat;
```

```
void main() {  
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;  
}
```

```
glLoadIdentity();  
double r = (double)getHeight()/getWidth();  
glFrustum(-1, 1, -r, r, 1, 20);  
glTranslated(0, 0, -3);  
glRotatef((float) now * 5.0f, 0, 0, 1);  
glRotatef(10 * (float) now * 5.0f, 1, 0, 0);  
glScaled(1.0f, 1.0f, 1.0f);
```

```
glLoadIdentity();  
double r = (double)getHeight()/getWidth();  
glFrustum(-1, 1, -r, r, 1, 20);  
glTranslated(0, 0, -3);  
glRotatef((float) now * 5.0f, 0, 0, 1);  
glRotatef(10 * (float) now * 5.0f, 1, 0, 0);  
glUniform3f(uniform_scale_mat, 1.0f, 1.0f, 1.0f);
```

MVP-Matrix erweitern in OpenGL

Mit folgendem Vertex-Shader bringen wir Bewegung ins Spiel:

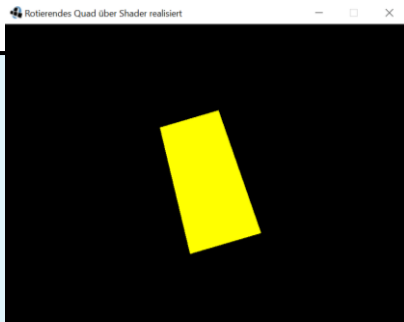
```
uniform vec3 uniform_scale_mat;

void main() {
    gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;
}
```

```
uniform vec3 uniform_scale_mat;

mat4 ScaleMatrix(const in vec3 s){
    return mat4(s.x, 0.0, 0.0, 0.0,
                0.0, s.y, 0.0, 0.0,
                0.0, 0.0, s.z, 0.0,
                0.0, 0.0, 0.0, 1.0);
}

void main() {
    gl_Position = gl_ModelViewProjectionMatrix *
        ScaleMatrix(uniform_scale_mat) * gl_Vertex;
}
```



```
glLoadIdentity();
double r = (double)getHeight()/getWidth();
glFrustum(-1, 1, -r, r, 1, 20);
glTranslated(0, 0, -3);
glRotatef((float) now * 5.0f, 0, 0, 1);
glRotatef(10 * (float) now * 5.0f, 1, 0, 0);
glUniform3f(uniform_scale_mat, 1.0f, 1.0f, 1.0f);
```

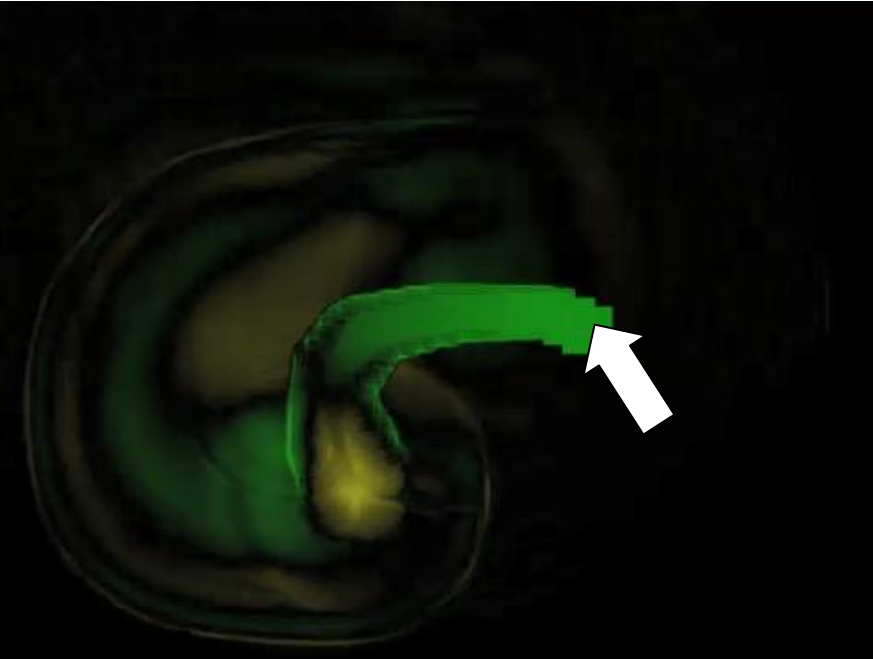
```
glLoadIdentity();
double r = (double)getHeight()/getWidth();
glFrustum(-1, 1, -r, r, 1, 20);
glTranslated(0, 0, -3);
glRotatef((float) now * 5.0f, 0, 0, 1);
glRotatef(10 * (float) now * 5.0f, 1, 0, 0);
glUniform3f(uniform_scale_mat, 0.5f, 1.0f, 1.0f);
```



Kommen wir jetzt wieder zu unserem Projekt

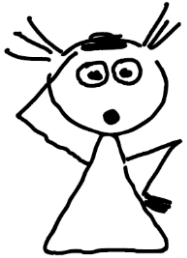
Projektziel – Stufe 3

In der dritten Projektstufe wollen wir die Wasserdynamik in Java realisieren. Wellenberge sollen gelb und –täler grün dargestellt werden. Wir verwenden auch hier wieder das Viereck als Objekt.



Projektstufe 3

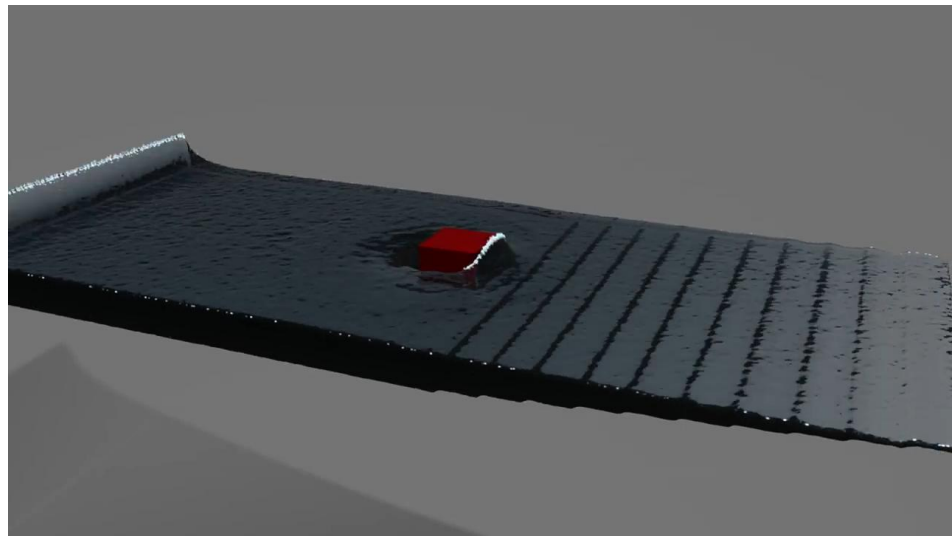
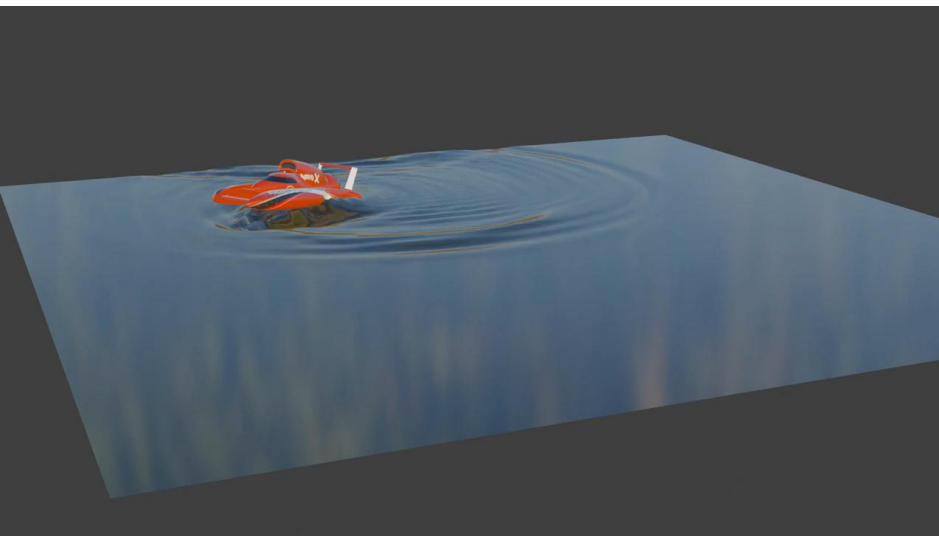
- Ersetzen des Blur-Effekts um Wasserdynamik
- Wellenberge sollen gelb und die Wellentäler grün visualisiert werden



Realisierung einer einfachen **Schwingungsdynamik**

Motivation für Wellengleichungen und Partikelsimulationen

Realistische Wassersimulationen kommen in vielen 3D-Welten zum Einsatz. Eine Interaktion zwischen Objekten und der **Wellendynamik** ist dabei obligatorisch (links, [1]). Hier sind die Lösungen oft einfach und in Echtzeit zu berechnen. **Fluid-Simulationen** in Kombination mit **Partikelsimulationen** sind weitaus realistischer (rechts, [2]), erfordern aber oft ein Offline-Rendering.



[1] <https://www.youtube.com/watch?v=gPcyYIA9HzU>

[2] <https://www.youtube.com/watch?v=Bljj3Qcmbf4>

Wellengleichung aus der Physik

Zweidimensionale Poisson-Gleichung und partielle Ableitung nach $\mathbf{t}$:

$$\frac{\partial^2 u(x, y)}{\partial x^2} + \frac{\partial^2 u(x, y)}{\partial y^2} = f(x, y) \qquad c^2 \left(\frac{\partial^2 y}{\partial x^2} + \frac{\partial^2 y}{\partial z^2} \right) = \frac{\partial^2 y}{\partial t^2}$$

Einsatz der Taylor-Formel zur Diskretisierung:

$$f(x) = f(a) + \frac{f'(a)}{1!}(x-a) + \dots + \frac{f^{(n)}(a)}{n!}(x-a)^n + R_{n+1}(x-a)$$

Ergibt zunächst:

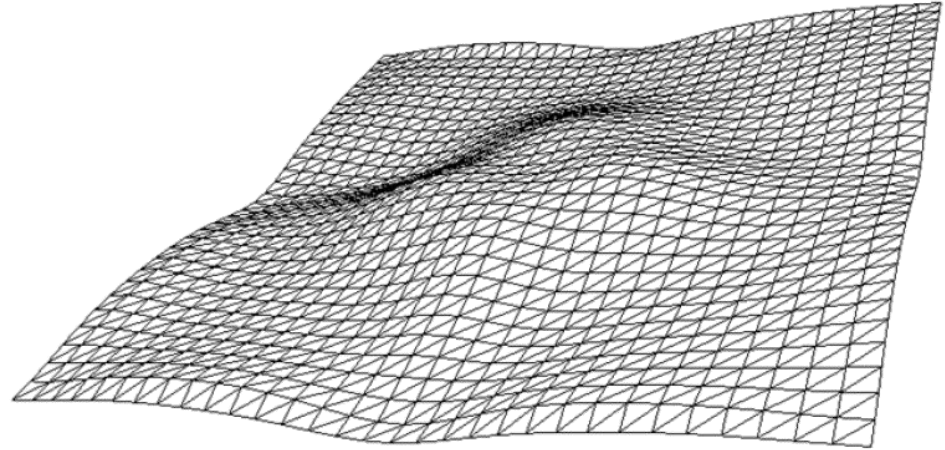
$$\left(\frac{\partial^2 y}{\partial h^2} \right)_{i,j} = \frac{y_{i+1,j} - 2y_{i,j} + y_{i-1,j}}{(\Delta h)^2} + O(\Delta h)^2$$

Jetzt noch einsetzen:

$$\frac{y_{i,j}^{t+1} - 2y_{i,j}^t + y_{i,j}^{t-1}}{(\Delta t)^2} = c^2 \left(\frac{y_{i+1,j}^t - 2y_{i,j}^t + y_{i-1,j}^t}{(\Delta x)^2} + \frac{y_{i,j+1}^t - 2y_{i,j}^t + y_{i,j-1}^t}{(\Delta z)^2} \right)$$

Als reguläres Gitter und über Crank-Nicholson-Verfahren:

$$\frac{y_{i,j}^{t+1} - 2y_{i,j}^t + y_{i,j}^{t-1}}{(\Delta t)^2} = c^2 \left(\frac{(y_{i+1,j}^{t+1} + y_{i+1,j}^t) + (y_{i-1,j}^{t+1} + y_{i-1,j}^t) + (y_{i,j+1}^{t+1} + y_{i,j+1}^t) + (y_{i,j-1}^{t+1} + y_{i,j-1}^t) - 4(y_{i,j}^{t+1} + y_{i,j}^t)}{2(\Delta h)^2} \right)$$



Es fehlt allerdings
noch die Dämpfung.

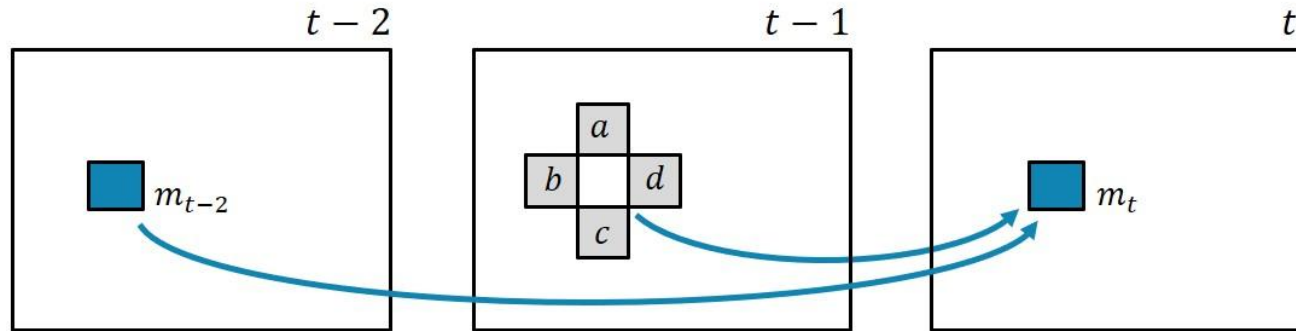


Keine partiellen Differentialgleichungen
zweiter Ordnung, sondern eine Lösung, die an den Blur-
Effekt erinnert.

Vereinfachte Schwingungsdynamik

Wir sehen hier eine kontinuierliche [Schwingungsdynamik](#), die einen wasserähnlichen Welleneffekt erzeugt. Für die Bestimmung eines Pixels im Zielbild zum Zeitpunkt t werden die Nachbarpixel im Vorgängerbild $t-1$ aufsummiert, halbiert und davon das Pixel aus dessen Vorgängerbild $t-2$ abgezogen. Basierend auf dieser Dynamik entsteht ein wellenähnlicher Effekt.

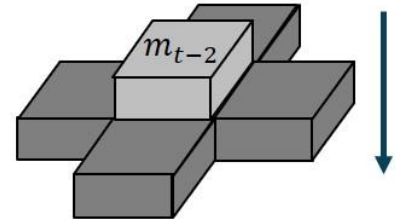
$$m_t = ((a + b + c + d) \cdot 0.5 - m_{t-2}) \cdot D$$



Zentrales Pixel von
Zeitpunkt $t-2$ wird ...

... von der Hälfte der Summe
der Nachbarn subtrahiert.

Das liefert den Pixelwert
im Zielbild

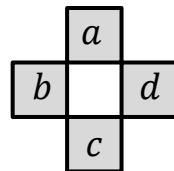


Pseudocode für vereinfachte Schwingungsdynamik

Der Pseudocode für die [Wasserdynamik](#) ähnelt sehr stark unserer Lösung vom gedämpften Mittelwertfilter.

```
for all target pixel (x, y) {  
    // Kernel für 4-er Nachbarschaft  
    target[x, y] = source[x-1, y] +  
                  source[x+1, y] +  
                  source[x, y-1] +  
                  source[x, y+1];  
  
    // Normalisierung und Zeitversatz  
    target[x, y] = target[x, y] * 0.5f - source2[x, y];  
  
    // Dämpfung  
    target[x, y] = target[x, y] * D;  
}
```

$$m_t = (a + b + c + d)$$



$$m_t = (a + b + c + d) \cdot 0.5 - m_{t-2}$$

$$m_t = ((a + b + c + d) \cdot 0.5 - m_{t-2}) \cdot D$$

Implementierung in Java

Die Implementierung könnte analog zum gedämpften Mittelwertfilter wie folgt aussehen:

```
public void renderLoop() throws InterruptedException {
    while (f.isVisible()) {
        float[] newC = surfaceA;
        float[] newB = surfaceC;
        float[] newA = surfaceB;

        surfaceA = newA;
        surfaceB = newB;
        surfaceC = newC;

        // water effect
        for (int y=1,o=WIDTH+1; y<HEIGHT-1; y++,o+=2)
            for (int x=1; x<WIDTH-1; x++,o++)
                surfaceC[o] = Math.min(1.0f, Math.max(-1.0f, D *
                    ((surfaceB[o-1]+surfaceB[o+1]+surfaceB[o-WIDTH]+
                     surfaceB[o+WIDTH])*0.5f -surfaceA[o]))));

        // Quaderobjekt
        int Q_SIZE = WIDTH/20;
        int posX = Math.min(Math.max(mouseCoords.x-Q_SIZE/2,1),
            WIDTH-Q_SIZE);
        int posY = Math.min(Math.max(mouseCoords.y-Q_SIZE/2,1),
            HEIGHT-Q_SIZE);
    }
}
```

```
// Alle Pixel des Quaderobjekts vollständig setzen
for (int y=0,o=posX+posY*WIDTH, r=WIDTH-Q_SIZE;
    y<Q_SIZE;y++,o+=r)
    for (int x=0; x<Q_SIZE; x++,o++)
        surfaceB[o] = 1;

for (int i=0,l=pixels.length;i<l;i++)
    pixels[i] = colorForAmplitude(surfaceC[i]);

mis.newPixels();
Thread.sleep(10);
}
```

```
public static int colorForAmplitudeGray(float a) {
    int gray = (int) ((a < 0 ? 0 : a) * 128);
    gray = gray > 128 ? 128 : gray;
    gray += 127;
    return 0xFF000000 | (gray << 16) | (gray << 8) | gray;
}
```

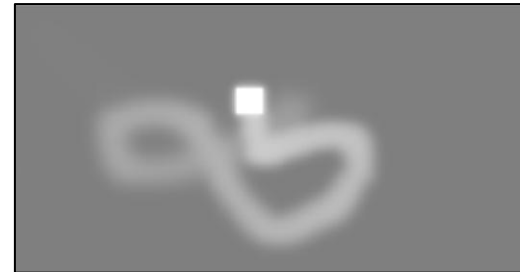


Da gab es doch eine kleine Übungsaufgabe ...

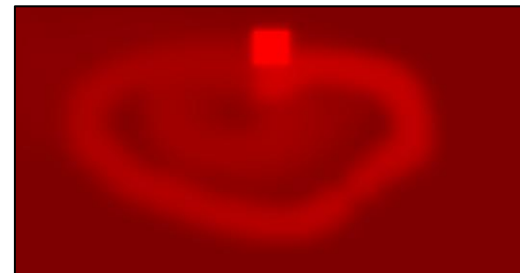
Lösung einer Übungsaufgabe I

Verändern Sie das Programm `BlurEffektCPU.java` so, dass nicht ein weißer Quader, sondern ein **roter Quader** gezeichnet wird.

```
public static int colorForAmplitudeGray(float a) {  
    int gray = (int) ((a < 0 ? 0 : a) * 128);  
    gray = gray > 128 ? 128 : gray;  
    gray += 127;  
    return 0xFF000000 | (gray << 16) | (gray << 8) | gray;  
}
```



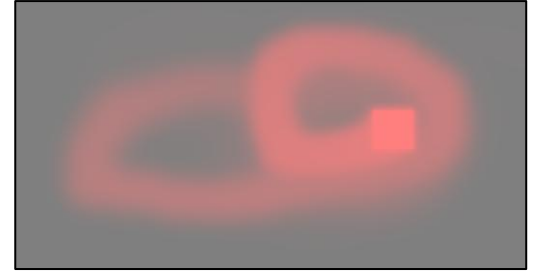
```
public static int colorForAmplitudeGray(float a) {  
    int gray = (int) ((a < 0 ? 0 : a) * 128);  
    gray = gray > 128 ? 128 : gray;  
    gray += 127;  
    return 0xFF000000 | (gray << 16);  
}
```



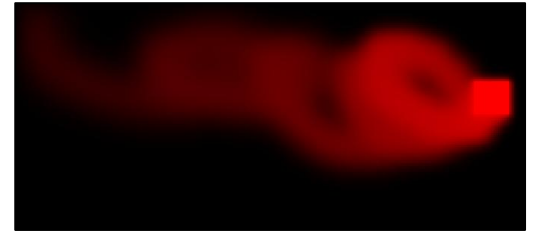
Lösung einer Übungsaufgabe II

Verändern Sie das Programm **BlurEffektCPU.java** so, dass nicht ein weißer Quader, sondern ein roter Quader gezeichnet wird.

```
public static int colorForAmplitudeGray(float a) {  
    int gray = (int) ((a < 0 ? 0 : a) * 128);  
    gray = gray > 128 ? 128 : gray;  
    gray += 127;  
    return 0xFF000000 | (gray << 16) | (127 << 8) | 127;  
}
```



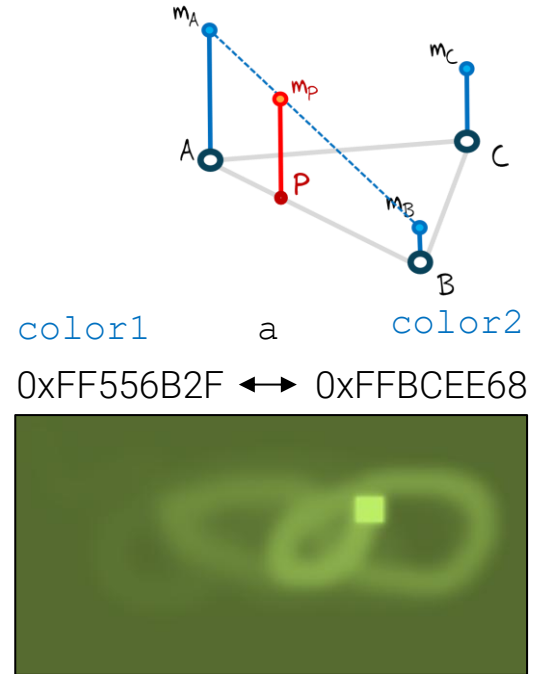
```
public static int colorForAmplitudeRed(float a) {  
    int red = (int) ((a < 0 ? 0 : a) * 255);  
    red = red > 255 ? 255 : red;  
    return 0xFF000000 | (red << 16);  
}
```



Lösung einer Übungsaufgabe III

Verändern Sie das Programm `BlurEffektCPU.java` so, dass nicht ein weißer Quader, sondern ein roter Quader gezeichnet wird.

```
public static int interpolateColor(float a, int color1, int color2) {  
    // Clamp amplitude to [0, 1]  
    if (a < 0) a = 0;  
    if (a > 1) a = 1;  
  
    // Farbkanäle aus color1 (Startfarbe) extrahieren  
    int r1 = (color1 >> 16) & 0xFF;  
    int g1 = (color1 >> 8) & 0xFF;  
    int b1 = color1 & 0xFF;  
  
    // Farbkanäle aus color2 (Ziel-Farbe) extrahieren  
    int r2 = (color2 >> 16) & 0xFF;  
    int g2 = (color2 >> 8) & 0xFF;  
    int b2 = color2 & 0xFF;  
  
    // Lineare Interpolation pro Kanal  
    int r = (int) (r1 + a * (r2 - r1));  
    int g = (int) (g1 + a * (g2 - g1));  
    int b = (int) (b1 + a * (b2 - b1));  
  
    return 0xFF000000 | (r << 16) | (g << 8) | b;  
}
```

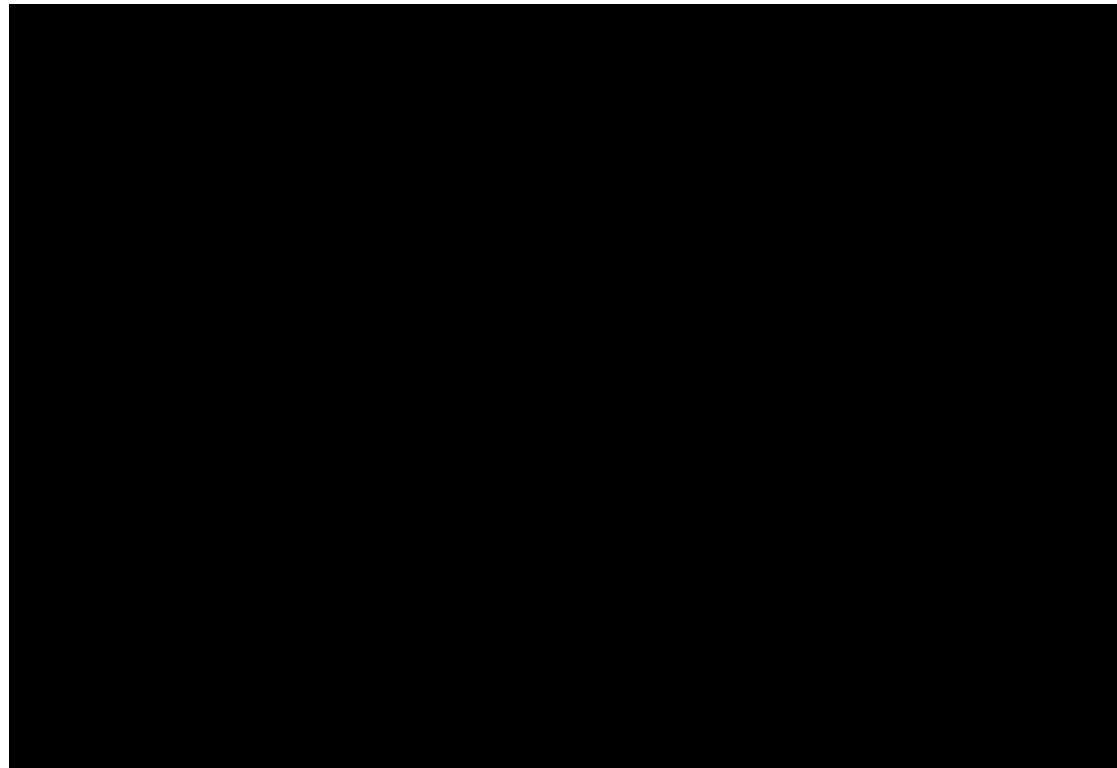




Nehmen wir die ursprüngliche (graue) Darstellung

Implementierung in Java

Wenn wir den Code direkt in Java abändern und darstellen, haben wir ein kleines Darstellungsproblem:

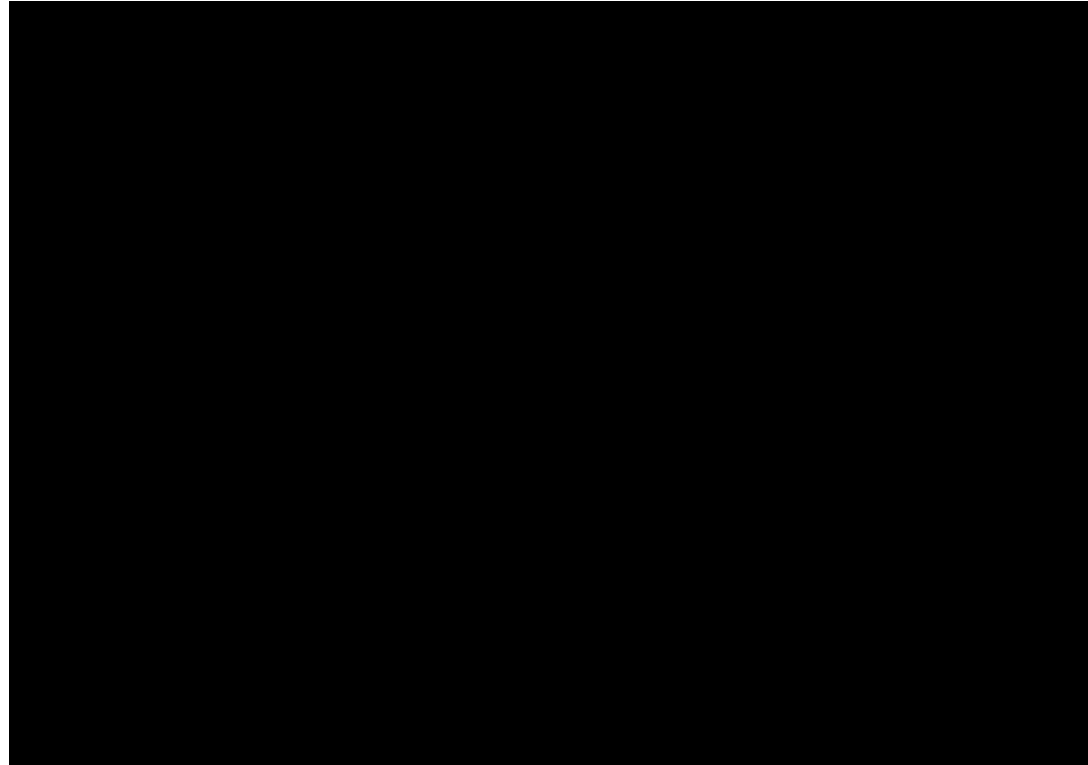




Wir haben **nur die Wellenberge** (positive Werte)
dargestellt!

Implementierung in Java (nach Korrektur)

Die unterschiedliche Darstellung von Wellenbergen und -tälern zeigt jetzt alle vorhandenen Daten:



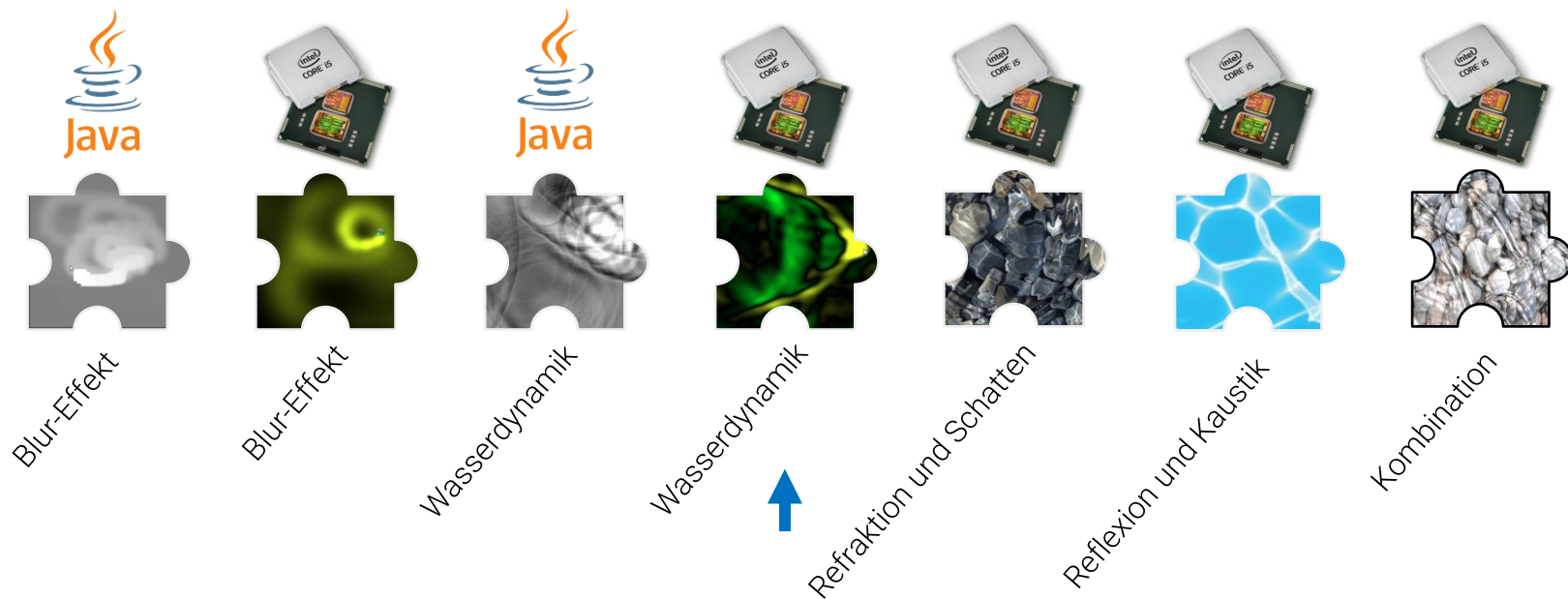
```
public static int colorForAmplitude(float a) {  
    int green = (int) ((a < 0 ? 0 : a) * 128);  
    int yellow = (int) ((-a < 0 ? 0 : -a) * 128);  
    green = green > 255 ? 255 : green;  
    yellow = yellow > 255 ? 255 : yellow;  
  
    return 0xFF000000 | (yellow << 16) |  
        (green << 8) | (yellow << 8);  
}
```



Programmcode:
kapitel02/WasserdynamikCPU.java

Interaktives Wasser – Projektetappen

Wasser hat in diesem Kontext schon immer eine besondere Anziehungskraft ausgeübt, symbolisiert es doch den Ursprung und das Grundbedürfnis des Lebens. Das folgende Projekt widmet sich dem Motto „Es war einmal das Wasser ...“ und wird Schritt für Schritt in die [Shaderprogrammierung mit GLSL](#) einführen und so – ganz nebenbei – die notwendigen mathematischen und physikalischen Konzepte behandeln. Beiläufig sehen wir instruktive Shaderbeispiele und machen uns mit GLSL vertraut.



Projektziel – Stufe 4

In der vierten Projektstufe wollen wir die Wasserdynamik im Shader realisieren. Wellenberge sollen gelb und –täler grün dargestellt werden.

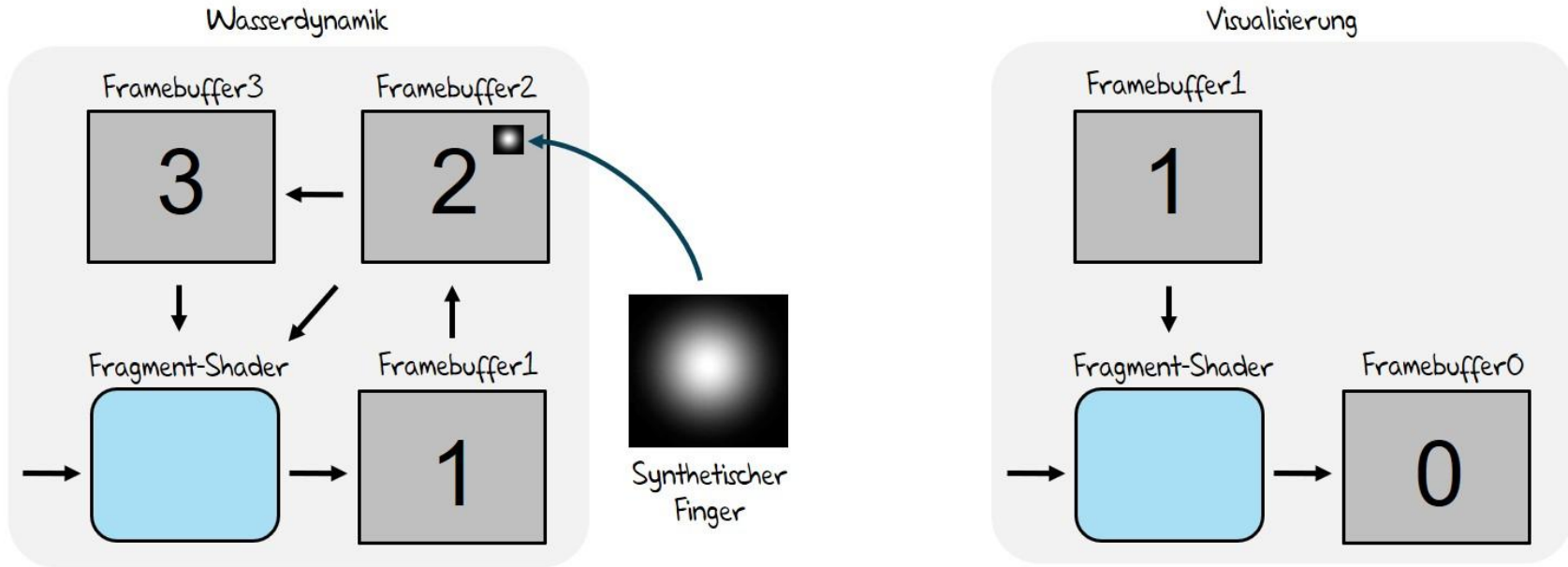


Projektstufe 4

- Ersetzen des Blur-Effekts um Wasserdynamik
- Wellenberge sollen gelb und die Wellentäler grün visualisiert werden

Shader-Strategie für die vereinfachte Schwingungsdynamik

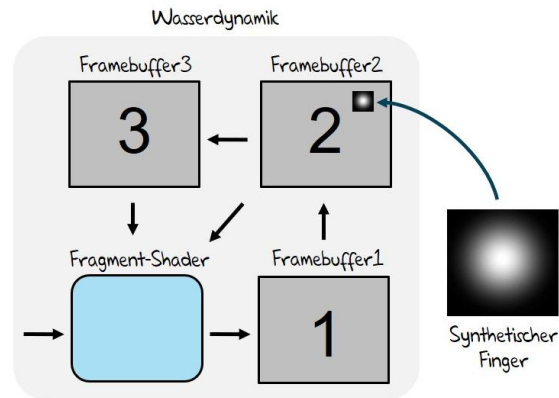
Hier findet die Verkettung von Wasserdynamik und Visualisierung statt. Zunächst verwenden wir die Renderpipeline in einem nicht sichtbaren Bereich für die Berechnung der Wasserdynamik. Anschließend wird das Ergebnis (Framebuffer1), bei dem sowohl **positive** als auch **negative Werte** vorkommen können, visuell interpretiert.



Implementierung Fragment-Shader Wasserdynamik

Schauen wir uns jetzt den kompletten Fragment-Shadercode für die [vereinfachte Wasserdynamik](#) an:

```
uniform sampler2D tex2;  
uniform sampler2D tex3;  
uniform vec2 s;  
  
void main (void) {  
    gl_FragColor = normalize((  
        texture2D(tex2, gl_TexCoord[0].st + vec2( s.x,    0)) +  
        texture2D(tex2, gl_TexCoord[0].st + vec2(-s.x,    0)) +  
        texture2D(tex2, gl_TexCoord[0].st + vec2(  0,  s.y)) +  
        texture2D(tex2, gl_TexCoord[0].st + vec2(  0, -s.y)))  
    ) * 0.5 - texture2D(tex3, gl_TexCoord[0].st)) * 0.992;  
}
```



$$m_t = ((a + b + c + d) \cdot 0.5 - m_{t-2}) \cdot D$$

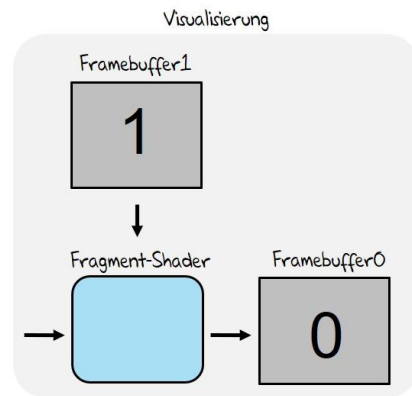
Implementierung Fragment-Shader Visualisierung

Schauen wir uns jetzt den kompletten Fragment-Shadercode für die [Visualisierung](#) an:

```
uniform sampler2D tex1;

void main (void) {
    vec4 col = texture2D(tex1, gl_TexCoord[0].st);
    col = (col.x <= 0) ? (col * -vec4(1.0, 1.0, 0.0, 1.0))
                      : (col *  vec4(0.0, 1.0, 0.0, 1.0));

    col *= 5.0;
    gl_FragColor = col;
}
```

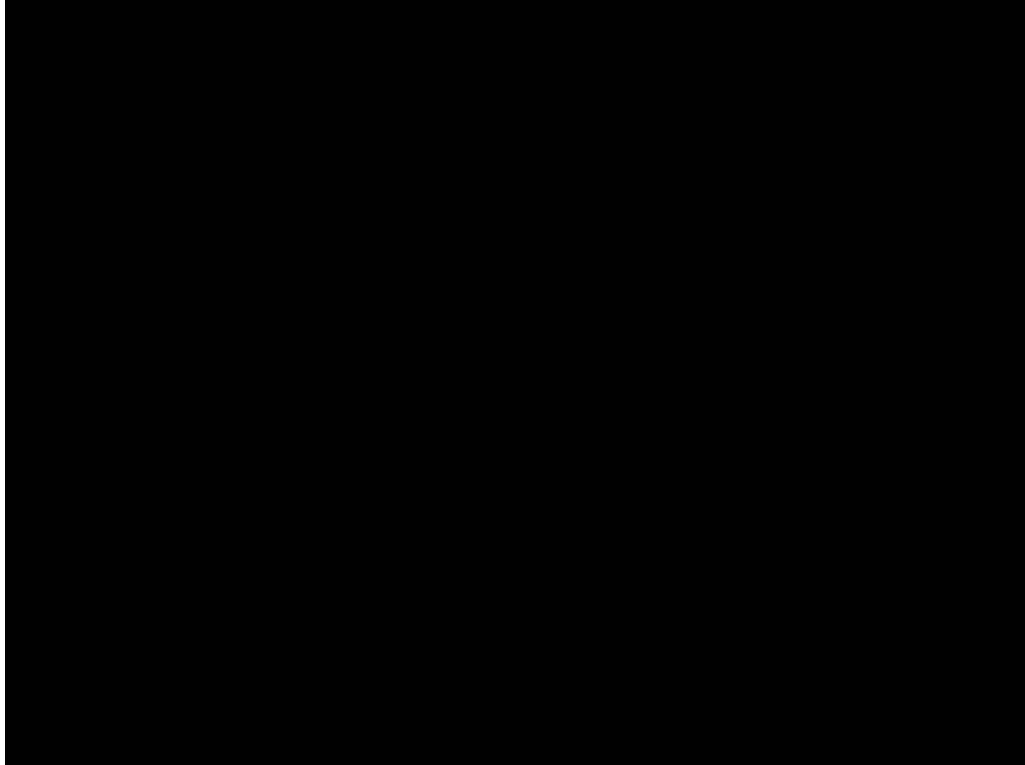




Schauen wir uns das Ergebnis an.

Implementierung im Shader

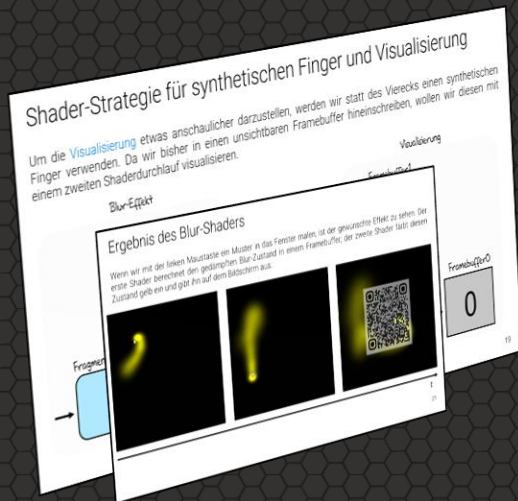
Die unterschiedliche Darstellung von Wellenbergen und -tälern zeigt jetzt alle vorhandenen Daten:



Programmcode:
kapitel02/WasserdynamikGPU.java

Realisierung einer einfachen Wasserdynamik in Java und im Shader

Key Takeaways

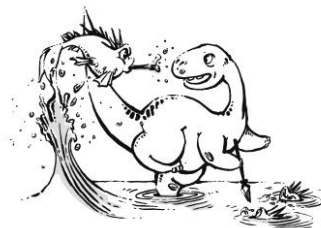


- Wasserdynamik über Nachbarschaft + Zeitversatz
- Unterschied zu Blur: Schwingung statt Glättung
- Nutzung mehrerer Zustände (t , $t-1$, $t-2$)
- Dämpfung verhindert Aufschaukeln
- CPU: Arrays + Schleifen (sequenziell)
- Shader: Texturen + Fragment-Shader (parallel)
- Trennung von Simulation und Visualisierung
- positive / negative Werte getrennt darstellen

Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

- (1) Wie lässt sich eine einfache Wasserdynamik über Konvolution realisieren? Ist das Vorgehen geeignet, um es in einem Shader berechnen zu lassen? Wenn ja, im Vertex- oder Fragmentshader?



Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

Aufgabe 1) [Java] Verändern Sie das Programm **WasserdynamikCPU.java** so, dass Wellenberge hellblau und Wellentäler dunkelblau dargestellt werden. Die mittlere Farbe sollte einem mittleren blau entsprechen.

Aufgabe 2) [Java] Für dieses Beispiel haben wir die Skalierungsmatrix als Funktion in den Shadercode übernommen:

```
glLoadIdentity();  
double r = (double)getHeight()/getWidth();  
glFrustum(-1, 1, -r, r, 1, 20);  
glTranslated(0, 0, -3);  
glRotatef((float) now * 5.0f, 0, 0, 1);  
glRotatef(10 * (float) now * 5.0f, 1, 0, 0);  
glScaled(1.0f, 1.0f, 1.0f);
```

Versuchen Sie das Gleiche mit der letzten Rotationsmatrix um die x-Achse.



Vielen Dank für die Aufmerksamkeit